

```

29 #include<stdio.h>
30 char* cutThemAll(int lengths_count, long *lengths, long minLength) {
31     long t=0,i=1;
32     for(int i=0;i<=lengths_count-1;i++){
33         t+=lengths[i];
34     }
35     do{
36         if(t-lengths[lengths_count-1]<minLength){
37             return "Impossible";
38         }
39         i++;
40     }while(i<lengths_count-i);
41     return "Possible";
42 }
43
44

```

	Test	Expected	Got	
✓	long lengths[] = {3, 5, 4, 3}; printf("%s", cutThemAll(4, lengths, 9))	Possible	Possible	✓
✓	long lengths[] = {5, 6, 2}; printf("%s", cutThemAll(3, lengths, 12))	Impossible	Impossible	✓

Passed all tests! ✓

```

9
10 /*
11  * To return the string from the function, you should either do static allocation or dynamic allocation
12  *
13  * For example,
14  * char* return_string_using_static_allocation() {
15  *     static char s[] = "static allocation of string";
16  *
17  *     return s;
18  * }
19  *
20  * char* return_string_using_dynamic_allocation() {
21  *     char* s = malloc(100 * sizeof(char));
22  *
23  *     s = "dynamic allocation of string";
24  *
25  *     return s;
26  * }
27  *
28  */
29 #include<stdio.h>
30 char* cutThemAll(int lengths_count, long *lengths, long minLength) {
31     long t=0,i=1;
32     for(int i=0;i<=lengths_count-1;i++){
33         t+=lengths[i];
34     }
35     do{
36         if(t-lengths[lengths_count-1]<minLength){
37             return "Impossible";
38         }
39         i++;
40     }while(i<lengths_count-i);
41     return "Possible";
42 }
43
44

```

STDIN Function

3 → lengths[] size n = 3
5 → lengths[] = [5, 6, 2]
6
2
12 → minLength= 12

Sample Output

Impossible

Explanation

The uncut rod is $5 + 6 + 2 = 13$ units long. After making either cut, the rod will be too short to make the second cut.

Answer: (penalty regime: 0 %)

Reset answer

1	/*	
2	* Complete the 'cutThemAll' function below.	
3	*	
4	* The function is expected to return a STRING.	
5	* The function accepts following parameters:	
6	* 1. LONG_INTEGER_ARRAY lengths	
7	* 2. LONG_INTEGER minLength	

Sample Case 0

Sample Input For Custom Testing

STDIN	Function
4	→ lengths[] size n = 4
3	→ lengths[] = [3, 5, 4, 3]
5	
4	
3	
9	→ minLength= 9

Sample Output

Possible

Explanation

The uncut rod is $3 + 5 + 4 + 3 = 15$ units long. Cut the rod into lengths of $3 + 5 + 4 = 12$ and 3 . Then cut the 12 unit piece into lengths 3 and $5 + 4 = 9$. The remaining segment is $5 + 4 = 9$ units and that is long enough to make the final cut.

Sample Case 1

Sample Input For Custom Testing

cutThemAll has the following parameter(s):

int lengths[n]: the lengths of the segments, in order

int minLength: the minimum length the machine can accept

Returns

string: "*Possible*" if all $n-1$ cuts can be made. Otherwise, return the string "*Impossible*".

Constraints

- $2 \leq n \leq 10^5$
- $1 \leq t \leq 10^9$
- $1 \leq lengths[i] \leq 10^9$
- The sum of the elements of *lengths* equals the uncut rod length.

Input Format For Custom Testing

The first line contains an integer, n , the number of elements in *lengths*.

Each line i of the n subsequent lines (where $0 \leq i < n$) contains an integer, *lengths*[i].

The next line contains an integer, *minLength*, the minimum length accepted by the machine.

the necessary cuts using this machine. The rod is marked into lengths already, in the order given.

Example

$n = 3$

$lengths = [4, 3, 2]$

$minLength = 7$

The rod is initially $sum(lengths) = 4 + 3 + 2 = 9$ units long. First cut off the segment of length $4 + 3 = 7$ leaving a rod $9 - 7 = 2$. Then check that the length 7 rod can be cut into segments of lengths 4 and 3. Since 7 is greater than or equal to $minLength = 7$, the final cut can be made. Return "Possible".

Example

$n = 3$

$lengths = [4, 2, 3]$

$minLength = 7$

The rod is initially $sum(lengths) = 4 + 2 + 3 = 9$ units long. In this case, the initial cut can be of length 4 or $4 + 2 = 6$. Regardless of the length of the first cut, the remaining piece will be shorter than $minLength$. Because $n - 1 = 2$ cuts cannot be made, the answer is "Impossible".

Function Description

Complete the function *cutThemAll* in the editor below.

```

36 #include<stdlib.h>
37 int* reverseArray(int arr_count, int *arr, int *result_count) {
38     int* result =(int*)malloc(arr_count * sizeof(int));
39     if(result == NULL){
40         return NULL;
41     }
42     for (int i=0;i<arr_count;i++)
43     {
44         result[i]=arr[arr_count-i-1];
45     }
46     *result_count = arr_count;
47     return result;
48
49 }
50
51

```

	Test	Expected	Got	
✓	<pre> int arr[] = {1, 3, 2, 4, 5}; int result_count; int* result = reverseArray(5, arr, &result_count); for (int i = 0; i < result_count; i++) printf("%d\n", *(result + i)); </pre>	<pre> 5 4 2 3 1 </pre>	<pre> 5 4 2 3 1 </pre>	✓

Passed all tests! ✓

An automated cutting machine is used to cut rods into segments. The cutting machine can only hold a rod of *minLength* or more, and it can only make one cut at a time. Given the array *lengths[]* representing the desired lengths of each segment, determine if it is possible to make

Answer: (penalty regime: 0 %)

Reset answer

```
1  /*
2  * Complete the 'reverseArray' function below.
3  *
4  * The function is expected to return an INTEGER_ARRAY.
5  * The function accepts INTEGER_ARRAY arr as parameter.
6  */
7
8  /*
9  * To return the integer array from the function, you should:
10 *     - Store the size of the array to be returned in the result_count variable
11 *     - Allocate the array statically or dynamically
12 *
13 * For example,
14 * int* return_integer_array_using_static_allocation(int* result_count) {
15 *     *result_count = 5;
16 *
17 *     static int a[5] = {1, 2, 3, 4, 5};
18 *
19 *     return a;
20 * }
21 *
22 * int* return_integer_array_using_dynamic_allocation(int* result_count) {
23 *     *result_count = 5;
24 *
25 *     int *a = malloc(5 * sizeof(int));
26 *
27 *     for (int i = 0; i < 5; i++) {
28 *         *(a + i) = i + 1;
29 *     }
30 *
31 *     return a;
32 * }
33 *
34 */
```


5

Sample Output

5

4

2

3

1

Explanation

The input array is [1, 3, 2, 4, 5], so the reverse of the input array is [5, 4, 2, 3, 1].

Sample Case 1

Sample Input For Custom Testing

4

17

10

21

45

Sample Output

45

21

10

17

Explanation

The input array is [17, 10, 21, 45], so the reverse of the input array is [45, 21, 10, 17].

Given an array of integers, reverse the given array in place using an index and loop rather than a built-in function.

Example

arr = [1, 3, 2, 4, 5]

Return the array [5, 4, 2, 3, 1] which is the reverse of the input array.

Function Description

Complete the function *reverseArray* in the editor below.

reverseArray has the following parameter(s):

int arr[n]: an array of integers

Return

int[n]: the array in reverse order

Constraints

$1 \leq n \leq 100$

$0 < arr[i] \leq 100$

Input Format For Custom Testing

The first line contains an integer, *n*, the number of elements in *arr*.

Each line *i* of the *n* subsequent lines (where $0 \leq i < n$) contains an integer, *arr[i]*.

Sample Case 0

Sample Input For Custom Testing

5

1

3

2

4

```

14  for(int i=0;i<n;i++){
15      scanf("%d %d %d",&s[i].a,&s[i].b,&s[i].c);
16  }
17  for(int i=0;i<n;i++){
18      for(int j=0;j<n;j++){
19          if(i!=j){
20              if(vol(s[i].a,s[i].b,s[i].c)<vol(s[j].a,s[j].b,s[j].c)){
21                  struct sides temp=s[j];
22                  s[j]=s[i];
23                  s[i]=temp;
24              }
25          }
26      }
27  }
28  for(int i=0;i<n;i++){
29      printf("%d %d %d\n",s[i].a,s[i].b,s[i].c);
30  }
31  return 0;
32  }

```

	Input	Expected	Got	
✓	3 7 24 25 5 12 13 3 4 5	3 4 5 5 12 13 7 24 25	3 4 5 5 12 13 7 24 25	✓

Passed all tests! ✓

3 4 5

Sample Output 0

3 4 5

5 12 13

7 24 25

Explanation 0

The square of the first triangle is **84**. The square of the second triangle is **30**. The square of the third triangle is **6**. So the sorted order is the reverse one.

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 struct sides{
3     int a;
4     int b;
5     int c;
6 };
7 int vol(int a,int b,int c){
8     return a*b*c;
9 }
10 int main(){
11     int n;
12     scanf("%d",&n);
13     struct sides s[n];
14     for(int i=0;i<n;i++){
```

The best way to calculate a volume of the triangle with sides ***a***, ***b*** and ***c*** is Heron's formula:

$$S = \sqrt{p * (p - a) * (p - b) * (p - c)} \text{ where } p = (a + b + c) / 2.$$

Input Format

First line of each test file contains a single integer ***n***. ***n*** lines follow with ***a_i***, ***b_i*** and ***c_i*** on each separated by single spaces.

Constraints

$$1 \leq n \leq 100$$

$$1 \leq a_i, b_i, c_i \leq 70$$

$$a_i + b_i > c_i, a_i + c_i > b_i \text{ and } b_i + c_i > a_i$$

Output Format

Print exactly ***n*** lines. On each line print **3** integers separated by single spaces, which are ***a_i***, ***b_i*** and ***c_i*** of the corresponding triangle.

Sample Input 0

3

7 24 25

5 12 13

```

6  scanf("%d",&n);
7  scanf("%d",&n);
8  struct book arr[n];
9  for(int i=0;i<n;i++){
10 scanf("%d %d %d",&arr[i].l,&arr[i].w,&arr[i].h);
11 }
12 for(int i=0;i<n;i++){
13 if(arr[i].h<41){
14 printf("%d\n",arr[i].l*arr[i].w*arr[i].h);
15 }
16 }
17 return 0;
18 }

```

	Input	Expected	Got	
✓	4	125	125	✓
	5 5 5	80	80	
	1 2 40			
	10 5 41			
	7 2 42			

Passed all tests! ✓

You are given n triangles, specifically, their sides a_i , b_i and c_i . Print them in the same style but sorted by their areas from the smallest one to the largest one. It is guaranteed that all the areas are different.

5 5 5
1 2 40
10 5 41
7 2 42

Sample Output 0

125
80

Explanation 0

The first box is really low, only **5** feet tall, so it can pass through the tunnel and its volume is **$5 \times 5 \times 5 = 125$** .

The second box is sufficiently low, its volume is **$1 \times 2 \times 4 = 80$** .

The third box is exactly **41** feet tall, so it cannot pass. The same can be said about the fourth box.

Answer: (penalty regime: 0 %)

1	#include<stdio.h>	
2	struct book{	
3	int l,w,h;	
4	};	
5	int main(){	
6	int n;	

You are transporting some boxes through a tunnel, where each box is a parallelepiped, and is characterized by its length, width and height.

The height of the tunnel **41** feet and the width can be assumed to be infinite. A box can be carried through the tunnel only if its height is strictly less than the tunnel's height. Find the volume of each box that can be successfully transported to the other end of the tunnel. Note: Boxes cannot be rotated.

Input Format

The first line contains a single integer ***n***, denoting the number of boxes.

n lines follow with three integers on each separated by single spaces - ***length_i***, ***width_i*** and ***height_i*** which are length, width and height in feet of the ***i***-th box.

Constraints

$$1 \leq n \leq 100$$

$$1 \leq \textit{length}_i, \textit{width}_i, \textit{height}_i \leq 100$$

Output Format

For every box from the input which has a height lesser than **41** feet, print its volume in a separate line.

Sample Input 0

Reset answer

```
1  /*
2  * Complete the 'minDiff' function below.
3  *
4  * The function is expected to return an INTEGER.
5  * The function accepts INTEGER_ARRAY arr as parameter.
6  */
7  #include<stdio.h>
8  int compare(const void *a,const void *b){
9      return (*(int*)a - *(int*)b);
10 }
11 int minDiff(int arr_count, int* arr)
12 {
13     qsort(arr,arr_count,sizeof(int),compare);
14     int totaldiff=0;
15     for(int i=1;i<arr_count;i++){
16         totaldiff+= abs(arr[i]-arr[i-1]);
17     }
18     return totaldiff;
19 }
20
```

	Test	Expected	Got	
✓	int arr[] = {5, 1, 3, 7, 3}; printf("%d", minDiff(5, arr))	6	6	✓

Passed all tests! ✓

	Test	Expected	Got	
✓	int arr[] = {1,2,3,4,5}; printf("%d", arraySum(5, arr))	15	15	✓

Passed all tests! ✓

Given an array of n integers, rearrange them so that the sum of the absolute differences of all adjacent elements is minimized. Then, compute the sum of those absolute differences. Example $n = 5$ $arr = [1, 3, 3, 2, 4]$ If the list is rearranged as $arr' = [1, 2, 3, 3, 4]$, the absolute differences are $|1 - 2| = 1$, $|2 - 3| = 1$, $|3 - 3| = 0$, $|3 - 4| = 1$. The sum of those differences is $1 + 1 + 0 + 1 = 3$. Function Description Complete the function `minDiff` in the editor below. `minDiff` has the following parameter: `arr`: an integer array Returns: `int`: the sum of the absolute differences of adjacent elements Constraints $2 \leq n \leq 105$ $0 \leq arr[i] \leq 109$, where $0 \leq i < n$ Input Format For Custom Testing The first line of input contains an integer, n , the size of `arr`. Each of the following n lines contains an integer that describes `arr[i]` (where $0 \leq i < n$). Sample Case 0 Sample Input For Custom Testing STDIN Function ----- 5 $\rightarrow$ `arr[]` size $n = 5$ 5 $\rightarrow$ `arr[]` = [5, 1, 3, 7, 3] 1 3 7 3 Sample Output 6 Explanation $n = 5$ $arr = [5, 1, 3, 7, 3]$ If `arr` is rearranged as $arr' = [1, 3, 3, 5, 7]$, the differences are minimized. The final answer is $|1 - 3| + |3 - 3| + |3 - 5| + |5 - 7| = 6$. Sample Case 1 Sample Input For Custom Testing STDIN Function ----- 2 $\rightarrow$ `arr[]` size $n = 2$ 3 $\rightarrow$ `arr[]` = [3, 2] 2 Sample Output 1 Explanation $n = 2$ $arr = [3, 2]$ There is no need to rearrange because there are only two elements. The final answer is $|3 - 2| = 1$.

Answer: (penalty regime: 0 %)

Reset answer

```

1  /*
2   * Complete the 'minDiff' function below.
3   *
4   * The function is expected to return an INTEGER.
5   * The function accepts INTEGER_ARRAY arr as parameter.
6   */
7  #include<stdio.h>
8  int compare(const void *a,const void *b){
9      return (*(int*)a - *(int*)b);
10 }
11 int minDiff(int arr_count, int* arr)

```

Explanation 1

12 + 12 = 24.

Answer: (penalty regime: 0 %)

Reset answer

```
1  /*
2  * Complete the 'arraySum' function below.
3  *
4  * The function is expected to return an INTEGER.
5  * The function accepts INTEGER_ARRAY numbers as parameter.
6  */
7
8  int arraySum(int numbers_count, int *numbers)
9  {
10     int sum =0;
11     for(int i=0;i<numbers_count;i++){
12         sum=sum+numbers[i];
13     }
14     return sum;
15 }
16
```

Test	Expected	Got
------	----------	-----

4

5

Sample Output 0

15

Explanation 0

1 + 2 + 3 + 4 + 5 = 15.

Sample Case 1

Sample Input 1

STDIN Function

2 → numbers[] size n = 2

12 → numbers = [12, 12]

12

Sample Output 1

24

int: integer sum of the numbers array

Constraints

$1 \leq n \leq 10^4$
 $1 \leq \text{numbers}[i] \leq 10^4$

Input Format for Custom Testing

Input from stdin will be processed as follows and passed to the function.

The first line contains an integer n, the size of the array numbers.
Each of the next n lines contains an integer numbers[i] where $0 \leq i < n$.

Sample Case 0
Sample Input 0

STDIN	Function
-----	-----
5	→ numbers[] size n = 5
1	→ numbers = [1, 2, 3, 4, 5]
2	
3	

	Test	Expected	Got	
✓	<pre>int arr[] = {1,2,3,3}; printf("%d", balancedSum(4, arr))</pre>	2	2	✓

Passed all tests! ✓

Calculate the sum of an array of integers.

Example

numbers = [3, 13, 4, 11, 9]

The sum is $3 + 13 + 4 + 11 + 9 = 40$.

Function Description

Complete the function arraySum in the editor below.

arraySum has the following parameter(s):

int numbers[n]: an array of integers

Returns

- The first and last elements are equal to 1.
- Using zero based indexing, arr[1]=2 is the pivot between the two subarrays.
- The index of the pivot is 1.

Answer: (penalty regime: 0 %)

Reset answer

```
1  /*
2  * Complete the 'balancedSum' function below.
3  *
4  * The function is expected to return an INTEGER.
5  * The function accepts INTEGER_ARRAY arr as parameter.
6  */
7
8  int balancedSum(int arr_count, int* arr)
9  {
10     int totalsum = 0;
11     for(int i=0;i<arr_count;i++){
12         totalsum+=arr[i];
13     }
14     int leftsum = 0;
15     for(int i = 0;i<arr_count;i++){
16         int rightsum = totalsum - leftsum-arr[i];
17         if(leftsum==rightsum){
18             return i;
19         }
20         leftsum+=arr[i];
21     }
22     return 1;
23 }
24
```

2

Explanation 0

- The sum of the first two elements, $1+2=3$. The value of the last element is 3.
- Using zero based indexing, $arr[2]=3$ is the pivot between the two subarrays.
- The index of the pivot is 2.

Sample Case 1

Sample Input 1

STDIN Function Parameters

3 → arr[] size n = 3
1 → arr = [1, 2, 1]
2
1

Sample Output 1

1

Explanation 1

- $3 \leq n \leq 10^5$
- $1 \leq \text{arr}[i] \leq 2 \times 10^4$, where $0 \leq i < n$
- It is guaranteed that a solution always exists.

Input Format for Custom Testing

Input from stdin will be processed as follows and passed to the function.

The first line contains an integer n , the size of the array arr .

Each of the next n lines contains an integer, $\text{arr}[i]$, where $0 \leq i < n$.

Sample Case 0

Sample Input 0

STDIN Function Parameters

4 → $\text{arr}[]$ size $n = 4$

1 → $\text{arr} = [1, 2, 3, 3]$

2

3

3

Sample Output 0

Given an array of numbers, find the index of the smallest array element (the pivot), for which the sums of all elements to the left and to the right are equal. The array may not be reordered.

Example

`arr=[1,2,3,4,6]`

- the sum of the first three elements, $1+2+3=6$. The value of the last element is 6.
- Using zero based indexing, `arr[3]=4` is the pivot between the two subarrays.
- The index of the pivot is 3.

Function Description

Complete the function `balancedSum` in the editor below.

`balancedSum` has the following parameter(s):

`int arr[n]`: an array of integers

Returns:

`int`: an integer representing the index of the pivot

Constraints